

Outline

What and why

Numerical and transformation features

Aggregations and time-series

Feature selection

Automation, pitfalls, production

Wrap-up

What is feature engineering?

Three related-but-distinct activities on tabular data:

- ▶ **Preprocessing (L01c).** *Fix* the data: handle missing values, encode categoricals, scale numerics. Same columns, cleaner.
- ▶ **Feature engineering.** *Create* new columns from existing ones using domain knowledge or general patterns. New columns.
- ▶ **Feature selection.** *Choose* which features to keep. Reduces columns.

The classical-ML maxim: “Good features beat a better algorithm.” Most Kaggle winners and most production wins on tabular data come from feature engineering, not the model class.

Deep learning learns features from raw inputs (pixels, tokens, audio). For tabular data, hand-engineered features still dominate.

Predict-first: where does an ML project's time go?

An ML project has several stages: data collection, EDA, preprocessing, feature engineering, modeling, tuning, evaluation, deployment.

Predict-first: where does an ML project's time go?

An ML project has several stages: data collection, EDA, preprocessing, feature engineering, modeling, tuning, evaluation, deployment.

Predict: what fraction of project time is spent on data & features vs modeling & tuning?

Predict-first: where does an ML project's time go?

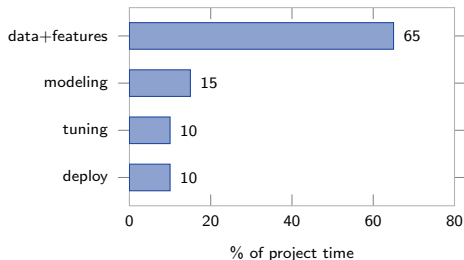
An ML project has several stages: data collection, EDA, preprocessing, feature engineering, modeling, tuning, evaluation, deployment.

Predict: what fraction of project time is spent on data & features vs modeling & tuning?

Surveys (CrowdFlower 2016, Anaconda 2020) consistently report:

- ▶ **60-80%** on data collection, cleaning, feature engineering.
- ▶ **10-20%** on modeling and tuning.
- ▶ **5-15%** on deployment and monitoring.

Students often expect the reverse. The reality: *the algorithm is the easy part*; the data preparation is the work.



Sources of new features

Where do good features come from?

1. **Domain knowledge.** The single biggest source. Examples: $\text{BMI} = \text{weight} / \text{height}^2$, loan-to-income ratio, price-per-m², click-through rate. A domain expert produces these in minutes; automated systems miss them.
2. **Numerical transformations.** Log, square root, binning, polynomial expansion.
3. **Combinations.** Ratios, differences, products, interactions.
4. **Aggregations.** Group-by statistics, rolling windows, expanding statistics.
5. **Temporal.** Lags, time-since-event, calendar features, cyclic encoding.
6. **Spatial.** Distances to landmarks, cluster IDs, grid cells.
7. **Text.** TF-IDF, n-grams, length, sentiment, embeddings.
8. **External data.** Joins to demographics, weather, prices, holidays.

The rest of this deck walks through each category with Yerevan apartment rent as the running example.

Numerical transformations

Log / power. For skewed positive targets (recap L01c).
`np.log1p(price)` compresses the tail.

Binning. Convert a continuous feature into categories.
Three strategies:

- ▶ Equal-width: same range per bin.
- ▶ Equal-frequency (quantile): same count per bin.
- ▶ Supervised: pick splits that maximize separation in y .

When to bin? When the target depends *non-monotonically* on the feature (e.g., rent peaks at 60 m² then plateaus) and you're using a linear model.
Trees don't need binning.

```
from sklearn.preprocessing \
    import KBinsDiscretizer

# 5 quantile bins on area
binner = KBinsDiscretizer(
    n_bins=5,
    encode="ordinal",
    strategy="quantile")
df["area_bin"] = binner.
    fit_transform(
        df[["area"]])
```



Polynomial and interaction features

Linear regression on *engineered* features is the most underrated trick in classical ML. From L01b: polynomial regression is OLS with $\phi(\mathbf{x}) = (1, x, x^2, \dots, x^d)$.

Interactions extend this to multiple features. Given x_1, x_2 :

$$\phi(\mathbf{x}) = (1, x_1, x_2, x_1^2, x_2^2, x_1x_2)$$

The product x_1x_2 lets a linear model express “effect of x_1 depends on level of x_2 .”

Predict-first: you expect rent to depend on `area × floor`. Do you need to add the product explicitly for: (a) Ridge regression, (b) Random Forest?

Answers: (a) **Yes** - a linear model can't construct products. (b) **No** - trees discover interactions via nested splits. Polynomial features are a *linear-model crutch*, not a universal good.

Ratios and differences

Many of the most predictive features are simple ratios or differences. Linear models can't construct them; trees can but slowly.

```
# Yerevan-rent examples
df["price_per_m2"] = df["rent"] /
    df["area"]
df["bedroom_density"] = df["rooms"] /
    df["area"]
df["age_at_listing"] = (df["
    year_listed"]
                        - df["
                            year_built
                        "])

# Classical examples
df["bmi"] = df["weight"] / df["
    height"]**2
df["lti"] = df["loan"] / df["income
    "]
df["debt_to_equity"] = df["debt"] /
    df["equity"]
```

Why ratios beat raw features:

- ▶ Often more predictive than either component alone.
- ▶ Express the underlying *rate* or *intensity*.
- ▶ Scale-invariant - a 3000 USD loan to a 6000 USD earner is the same as 30k/60k.
- ▶ Sparse - one engineered feature vs. two raw ones.

When NOT to engineer:

- ▶ If the denominator can be 0 or negative.
- ▶ For deep learning on tabular - just feed in the raw inputs.

Datetime features

Datetimes are rich. Extract components, time-since-events, calendar markers, and cyclic encodings.

```
df["dt"] = pd.to_datetime(df["listed_at"])
df["year"] = df["dt"].dt.year
df["month"] = df["dt"].dt.month
df["weekday"] = df["dt"].dt.dayofweek          # 0=Mon, 6=Sun
df["is_weekend"] = df["weekday"].isin([5, 6]).astype(int)
df["days_since_listed"] = (today - df["dt"]).dt.days
df["is_holiday"] = df["dt"].dt.date.isin(holiday_calendar)

# cyclic encoding for periodic features (recap L01c)
df["hour_sin"] = np.sin(2 * np.pi * df["hour"] / 24)
df["hour_cos"] = np.cos(2 * np.pi * df["hour"] / 24)
```

Watch for **leakage** when computing “time since last event” - use only events strictly before the row's timestamp.

Group-by aggregations

For each row, attach summary statistics of its group. Hugely effective for tabular ML.

```
# Yerevan: for each apartment, learn the district's price profile
df["avg_rent_district"] = df.groupby("district")["rent"].transform("mean")
df["med_rent_district"] = df.groupby("district")["rent"].transform("median")
df["std_rent_district"] = df.groupby("district")["rent"].transform("std")
df["count_in_district"] = df.groupby("district")["rent"].transform("count")
df["rank_in_district"] = df.groupby("district")["rent"].rank(pct=True)
df["rent_vs_district_avg"] = df["rent"] / df["avg_rent_district"]
```

Leakage warning: computing group means on the *full* dataset before splitting leaks the target into features. Either fit the aggregation on the train fold only, or use a CV-aware target encoder from `category_encoders`.

Time-series features: lags and rolling windows

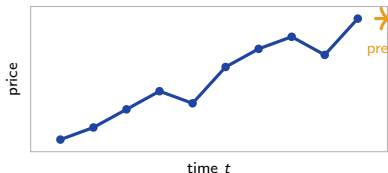
```
df = df.sort_values("dt")

# lag features
df["lag_1"] = df["price"].shift(1)
df["lag_7"] = df["price"].shift(7)

# rolling windows (trailing!)
df["ma_7"] = df["price"].rolling(7).mean()
df["std_7"] = df["price"].rolling(7).std()

# expanding / EMA
df["ema_7"] = df["price"].ewm(span=7).mean()

# diff features
df["diff_1"] = df["price"].diff(1)
df["pct_chg"] = df["price"].pct_change()
```



Look-ahead bias: `rolling(7, center=True)` silently uses future values. *Never center rolling windows when forecasting.*

Geospatial features

When you have lat/lon, the raw coordinates are usually useless; the *relationships* are predictive.

```
from math import radians, sin, cos, asin, sqrt

def haversine(lat1, lon1, lat2, lon2):
    R = 6371          # km
    dlat = radians(lat2 - lat1)
    dlon = radians(lon2 - lon1)
    a = (sin(dlat/2)**2
         + cos(radians(lat1))
         * cos(radians(lat2))
         * sin(dlon/2)**2)
    return 2 * R * asin(sqrt(a))

# Distance to Republic Square (Yerevan center)
df["dist_to_center"] = df.apply(lambda r:
    haversine(r.lat, r.lon, 40.177,
              44.512),
    axis=1)
```

Useful geospatial features:

- ▶ **Distances** to landmarks (metro, river, parks).
- ▶ **Cluster IDs** from k-means or DBSCAN on (lat, lon).
- ▶ **Grid cells** via geohash, H3, or S2 - aggregations within a cell.
- ▶ **Density features** (k-NN density estimate around each point).
- ▶ **POI counts** within a radius (cafes, schools, public transport).

Raw lat/lon as columns in a linear model: don't. Trees can split on lat/lon but distances/clusters are usually more efficient.

Text features (intro)

Text needs to become numbers. Three layers, increasing in power:

- ▶ **Engineered scalars.** Length, word count, punctuation count, ALL-CAPS ratio, emoji count. Cheap and surprisingly predictive for spam, sentiment.
- ▶ **Bag-of-words / TF-IDF / n-grams.** Tokenize, count, weight by inverse document frequency. The classical NLP workhorse.
- ▶ **Embeddings.** Word2Vec, sentence-BERT, modern transformer encoders. Deferred to the NLP chapter.

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer(max_features=2000, ngram_range=(1, 2),
                        min_df=5, stop_words="english")
X_text = tfidf.fit_transform(df["description"])

df["desc_length"] = df["description"].str.len()
df["caps_ratio"] = df["description"].str.count(r"[A-Z]") / df["desc_length"]
df["num_exclaim"] = df["description"].str.count(r"!")
```

Feature crosses for high-cardinality

When two high-cardinality categoricals matter *jointly*, you can either combine them into one feature (a “cross”) or learn an embedding.

```
# Manual cross: district x rooms -> Kentron_3br
df["district_x_rooms"] = df["district"] + "_" + df["rooms"].astype(str)

# Then encode the cross with target / frequency / OHE
```

- ▶ Used heavily in classical CTR prediction (Google's Wide & Deep paper, 2016).
- ▶ Risk: explosion. $100 \text{ districts} \times 10 \text{ room counts} = 1000 \text{ categories}$.
- ▶ Modern alternative: learn an *embedding* per categorical, sum or concatenate. NN does this naturally; `category_encoders` has `HashingEncoder` for the classical version.

Why select features at all?

More features is not always better.

- ▶ **Curse of dimensionality** (math module 30). As p grows, the volume of the feature space grows exponentially. You need exponentially more data to fill it.
- ▶ **Multicollinearity**. Redundant features make linear coefficients unstable and uninterpretable (L01b).
- ▶ **Overfitting**. The fewer features the simpler the model; the simpler the model the lower the variance.
- ▶ **Compute and storage**. Linear in p at inference time.
- ▶ **Interpretability**. “We use these 8 features” is easier to defend than “we use these 800”.

Three families of methods, in increasing computational cost:

1. **Filter** - score each feature independently.
2. **Embedded** - use a model that selects as it fits.
3. **Wrapper** - search over subsets, evaluate each.

Filter methods

Score each feature individually against the target. Pick the top k .

```
from sklearn.feature_selection import (  
    SelectKBest, f_regression, mutual_info_regression, VarianceThreshold)  
  
# Correlation-based (regression)  
SelectKBest(score_func=f_regression, k=20).fit(X, y)  
  
# Mutual information - catches non-linear relationships  
SelectKBest(score_func=mutual_info_regression, k=20).fit(X, y)  
  
# Drop near-constant features  
VarianceThreshold(threshold=0.01).fit(X)
```

- ▶ **Pros:** fast (linear in p), simple, model-agnostic.
- ▶ **Cons:** misses interactions. A feature useless alone may be predictive in combination.
- ▶ Best as a *first pass* on a $p > 10,000$ feature space.

Embedded and wrapper methods

Embedded. The model selects as part of fitting.

```
from sklearn.feature_selection \
    import SelectFromModel
from sklearn.linear_model import LassoCV

# L1 forces sparsity
lasso = LassoCV(cv=5).fit(X, y)
sel = SelectFromModel(lasso, prefit=True)
X_sel = sel.transform(X)
```

Lasso, ElasticNet, tree feature importances, RF permutation importance.

Rule of thumb: filter for initial cut, embedded (Lasso or tree importance) for production, wrapper only when stakes justify the compute.

Wrapper. Search subsets, score each with CV.

```
from sklearn.feature_selection \
    import SequentialFeatureSelector

sfs = SequentialFeatureSelector(
    Ridge(),
    n_features_to_select=10,
    direction="forward", # or "
                        backward"
    cv=5).fit(X, y)
```

Slow (fits the model $\mathcal{O}(p^2)$ times) but accurate.

Automated feature engineering

Tools that generate features automatically from your raw data:

- ▶ **featuretools**. “Deep Feature Synthesis” walks relational schemas and generates aggregations, transformations, and combinations.
- ▶ **tsfresh**. Extracts hundreds of statistical features from time-series.
- ▶ **autofeat**, **Featurewiz**. General-purpose automated FE.

```
import featuretools as ft
es = ft.EntitySet(id="rent")
es = es.add_dataframe(dataframe=df, dataframe_name="listings", index="id")
es = es.add_dataframe(dataframe=districts, dataframe_name="districts", ...)
feature_matrix, defs = ft.dfs(entityset=es,
                             target_dataframe_name="listings",
                             max_depth=2)
```

When to use: early exploration, complex relational data, generating candidates.

When NOT to use: production (hard to maintain, slow at inference). Auto-generated features usually *lose* to a handful of domain features.

Six feature-engineering pitfalls

1. **Target leakage.** Computing a target-derived feature (mean rent by district) on the *full* dataset before splitting. Use train fold only or a CV-aware encoder.
2. **Look-ahead bias.** For time-series, using future values in a “rolling” or aggregate feature. Always trailing windows.
3. **Train-test inconsistency.** A feature that’s easy to compute on training data but impossible at inference (e.g., “count of total purchases” when at inference you don’t know the user’s full history yet).
4. **Hardcoded statistics.** “Just divide by the mean rent” - whose mean? Save the fitted statistic, don’t recompute on test.
5. **Over-engineering.** 500 features on 1000 rows. Curse of dimensionality + multicollinearity. Stop when CV stops improving.
6. **Forgetting documentation.** What was feat_03? Six months from now you won’t know. Name features descriptively (rent_zscore_by_district).

Feature stores: serving the same features in training and inference

In production, the model you train and the model you serve see different code paths. Without care, they compute features differently.

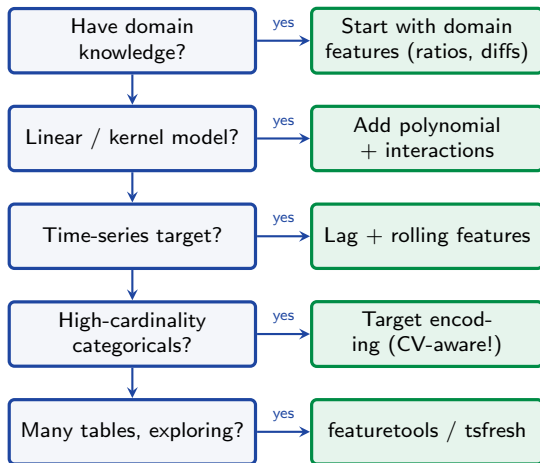
Feature store = a system that:

- ▶ Stores feature definitions (the code that computes each feature).
- ▶ Serves both *batch* (training queries against historical data) and *online* (low-latency lookup for inference).
- ▶ Guarantees train/serve consistency.
- ▶ Versions features over time.
- ▶ Handles point-in-time correctness for time-series (no leakage).

Production-grade tools: **Feast** (open source), **Tecton**, **Vertex AI Feature Store**, **SageMaker Feature Store**.

For coursework you don't need this; for any deployed model with a feedback loop, you do.

Decision flowchart



After engineering, always do a feature-selection pass (Lasso or tree importance) and check CV improvement before keeping.

Recap

- ▶ **60-80% of an ML project** is data and features, not modeling. The maxim “good features beat a better algorithm” holds for classical ML.
- ▶ **Domain knowledge is the multiplier.** Ratios (BMI, price-per-m²), differences (age, days-since), and rates beat raw features.
- ▶ **Transformations:** log/power for skew, binning for non-monotonic effects, polynomial + interactions for linear models (not for trees).
- ▶ **Aggregations:** group-by stats, rolling windows (trailing only), expanding stats. The leakage hides in the fit-on-full-data step.
- ▶ **Specialty domains:** datetime (cyclic, components), geospatial (distances, clusters), text (TF-IDF), high-cardinality crosses.
- ▶ **Selection:** filter → embedded → wrapper, in order of cost.
- ▶ **Automation tools** (featuretools, tsfresh) for exploration, not production. Domain features still win.
- ▶ **Pitfalls:** target leakage, look-ahead bias, train-serve skew. Wrap everything in a Pipeline (L01c) and serve from a feature store in production.

HW01g - practice

Estimated time: 90-120 minutes. Tasks 1-4 graded; bonuses optional.

1. On the Yerevan rent dataset, engineer 5 domain features by hand (`price_per_m2`, `age_at_listing`, `avg_rent_district`, `rent_vs_district_avg`, `dist_to_center`). Fit Ridge with and without. Report test MAE.
2. Add `PolynomialFeatures(degree=2)` to the original feature set. Compare CV score to step 1.
3. Fit Lasso on the union of original + engineered + polynomial features. Which features survive? Compare CV score to manual domain features only.
4. Replace Ridge with a tree-based model (e.g. `HistGradientBoostingRegressor`). Drop the polynomial features. Compare CV score to the linear pipeline.
5. *Bonus 1:* compute `avg_rent_district` the wrong way (on full data, then split). Report inflated vs honest CV scores.
6. *Bonus 2:* use `featuretools` on the dataset. Compare auto-generated features to your manual ones. Which set wins on test? **Most instructive question.**
7. *Bonus 3:* build a rolling 30-day average rent feature using listings sorted by date. Verify no look-ahead bias.